

Agenda

- Lambda Expressions
- Stream Programming

2. Functional Interfaces

- If interface contains exactly one abstract method (SAM), it is said to be functional interface.
- It may contain additional default & static methods. E.g. Comparator, Runnable, ...
- @FunctionalInterface annotation does compile time check, whether interface contains single abstract method. If not, raise compile time error.
- Functional interfaces forms foundation for Java lambda expressions.

Built-in functional interfaces

- New set of functional interfaces given in java.util.function package.
 - `Predicate<T>`: test: T -> boolean
 - `Function<T,R>`: apply: T -> R
 - `BiFunction<T,U,R>`: apply: (T,U) -> R
 - `UnaryOperator<T>`: apply: T -> T
 - `BinaryOperator<T>`: apply: (T,T) -> T
 - `Consumer<T>`: accept: T -> void
 - `Supplier<T>`: get: () -> T
- For efficiency primitive type functional interfaces are also supported e.g. IntPredicate, IntConsumer, IntSupplier, IntToDoubleFunction, ToIntFunction, ToIntBiFunction, IntUnaryOperator, IntBinaryOperator.

Lambda expressions

- Traditionally Java uses anonymous inner classes to compact the code. For each inner class separate .class file is created.
- However code is complex to read and un-efficient to execute.
- Lambda expression is short-hand way of implementing functional interface.
- Its argument types may or may not be given. The types will be inferred.
- Lambda expression can be single liner (expression not statement) or multi-liner block { ... }.
- Practically lambda expressions are used to pass as argument to various functions.
- Lambda expression enable developers to write concise code (single liners recommended).

Java 8 Streams

- Java 8 Stream is NOT IO streams.
- java.util.stream package.
- Streams follow functional programming model in Java 8.
- The functional programming is based on functional interface (SAM).
- Number of predefined functional interfaces added in Java 8. e.g. Consumer, Supplier, Function, Predicate, ...
- Lambda expression is short-hand way of implementing SAM -- arg types & return type are inferred.
- Java streams represents pipeline of operations through which data is processed.

- Stream operations are of two types
 1. Intermediate operations: Yields another stream.
- intermediate operations are again classified as
 - filter(), map(), sorted(), distinct()
- 2. Terminal operations: Yields some result.
- forEach()

Java stream characteristics

1. No storage: Stream is an abstraction. Stream doesn't store the data elements. They are stored in source collection or produced at runtime.
2. Immutable: Any operation doesn't change the stream itself. The operations produce new stream of results.
3. Lazy evaluation: Stream is evaluated only if they have terminal operation. If terminal operation is not given, stream is not processed.
4. Not reusable: Streams processed once (terminal operation) cannot be processed again.

Stream operations

- Source of elements

```
String[] names = {"Smita", "Rahul", "Rachana", "Amit", "Shraddha", "Nilesh",  
"Rohan", "Pradnya", "Rohan", "Pooja", "Lalita"};
```

- Create Stream and display all names

```
Stream.of(names)  
    .forEach(s -> System.out.println(s));
```

- filter() -- Get all names ending with "a"
 - `Predicate<T>: (T) -> boolean`

```
Stream.of(names)  
    .filter(s -> s.endsWith("a"))  
    .forEach(s -> System.out.println(s));
```

- map() -- Convert all names into upper case
 - `Function<T,R>: (T) -> R`

```
Stream.of(names)  
    .map(s -> s.toUpperCase())
```

```
.forEach(s -> System.out.println(s));
```

- sorted() -- sort all names in ascending order
 - String class natural ordering is ascending order.
 - sorted() is a stateful operation (i.e. needs all element to sort).

```
Stream.of(names)  
    .sorted()  
    .forEach(s -> System.out.println(s));
```

SUNBEAM